# FL-IDS: Federated Learning-Based Intrusion Detection System for IoT/IIoT


Aaron B Ajay
1BM23CS003

Aaryan Prakash
1BM23CS006

Amal Roy
1BM23CS025

Amogh Krishna JS
1BM23CS029

Mr. Syed Akram
Assistant Professor
Department of Computer Science and Engineering
B.M.S. College of Engineering

# OUTLINE

1. Problem Statement
2. High level Design
3. Detailed Design
4. Implementation
5. Results (Table, Graphs, ….Datasets Used (If Any)) and Snapshots of the output
6. Conclusion and Future Enhancements
7. Paper published
8. References

# Sustainable Development Goals (SDGs) Addressed

### SDG 9: Industry, Innovation and Infrastructure

- Enhances secure network infrastructure
- Supports smart systems

### SDG 11: Sustainable Cities and Communities

- Secure IoT systems for smart cities, including traffic systems, healthcare and energy grids
- Helps ensure safe and reliable operation of connected urban infrastructure

### SDG 16: Peace, Justice and Strong Institutions

- Strengthens cybersecurity
- Protects sensitive digital data

Promotes **secure and resilient digital ecosystems**

# Collaboration

## Category: Not a multidisciplinary project

A Computer Science and Engineering project; not community-oriented or multidisciplinary.

## Field work / collaboration

No external collaboration.

We use the public Edge-IIoTset dataset and its real packet captures, recorded on a seven-layer IoT/IIoT testbed with 10+ kinds of sensors (Ferrag et al., IEEE Access, 2022).

# Problem Statement

IoT and industrial IoT devices are everywhere, lightly protected, and attacked constantly.

### Privacy

Sending every site's raw traffic to one place exposes how a factory or building runs.

### New attacks

A normal classifier can only name attacks it was trained on.

### Sabotage

In shared training, a few bad participants can poison the model.

### No action

Detecting an attack does not stop it.

### Our goal

One system that solves all four, and classifies attacks with at least 95% accuracy and 0.95 macro F1.

# Objectives

### 1. Malicious Client Detection & Filtering

To develop a **malicious client detection mechanism** using anomaly-based or trust-based evaluation to identify and filter compromised participants in the federated network.

### 2. Efficient Privacy-Aware FL Design

To design a **lightweight and efficient federated learning framework** that balances privacy and performance while minimizing computational and communication overhead in resource-constrained IoT environments.

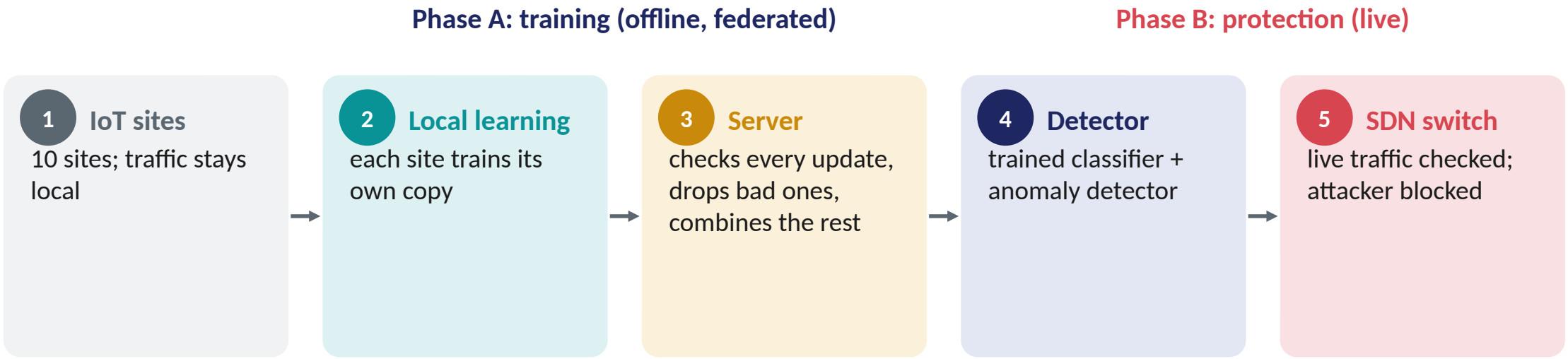
### 3. Model Optimization for Scalability

To enhance system scalability by applying **model optimization techniques such as weight pruning**, reducing computational and communication overhead.

### 4. Semi-Supervised & Anomaly-Based Detection

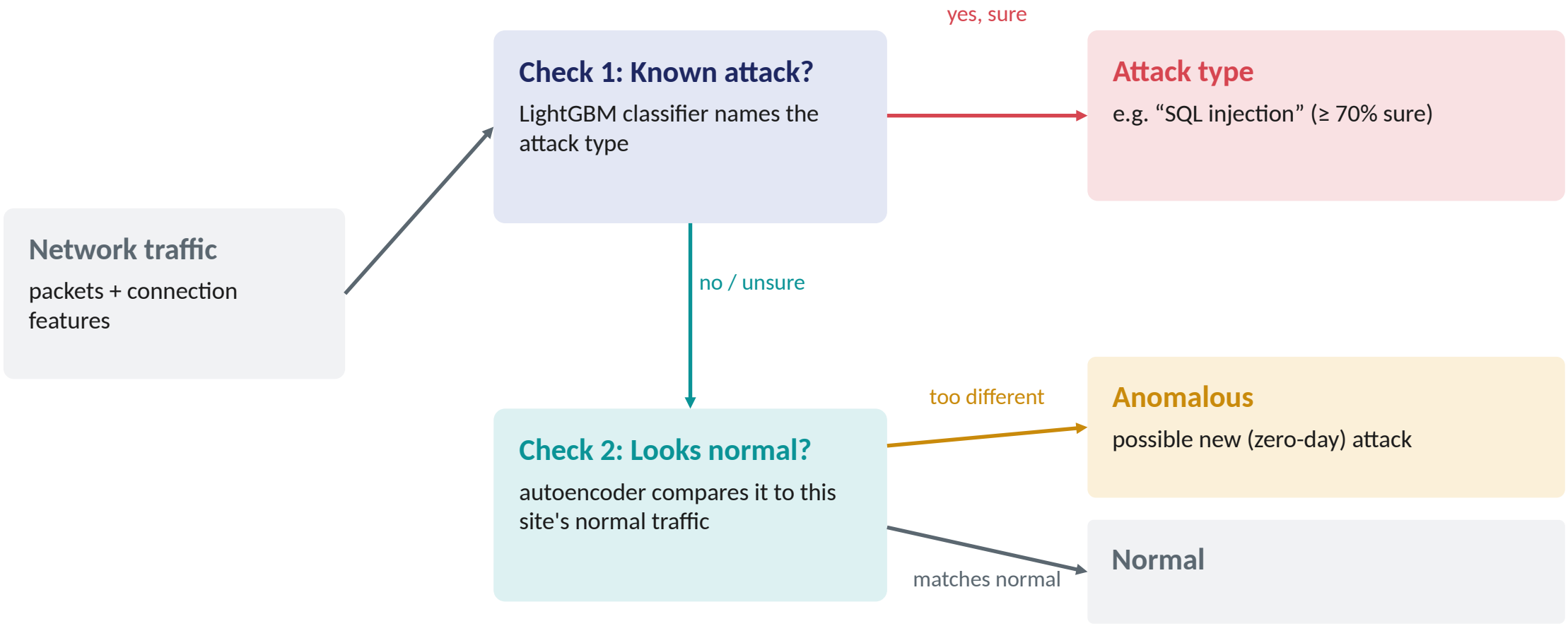
To integrate **semi-supervised learning and Autoencoder-based anomaly detection** to effectively identify abnormal or potentially poisoned patterns without relying entirely on labeled data.

# High level Design




Raw traffic never leaves a site: only model updates travel to the server. Phase A produces a saved model bundle; Phase B loads it on an emulated network (Mininet + Open vSwitch).

# Detailed Design: Two Checks per Packet



# Detailed Design: Training Safely Together

**1 Send model**
server sends the current detector to all 10 sites

→

**2 Learn locally**
each site trains on its own traffic, which never leaves

→

**3 Trust check**
server compares each site's change with the group's typical change

→

**4 Combine**
suspicious changes are dropped; the rest are averaged robustly

repeated for 20 rounds

### The trust check in one line

Like a teacher comparing homework: if one student's answer points the opposite way to the class's typical answer, or keeps disagreeing round after round, it is set aside. Large answers are also capped so one student can't dominate.

# Implementation

| Part | Built with |
|---|---|
| Data preparation | Python, pandas: cleaning, dataset repairs, new features |
| Check 1: classifier | LightGBM (gradient-boosted trees) |
| Check 2: anomaly detector | PyTorch autoencoder, one per site |
| Federated training | Flower: 1 server + 10 real client processes |
| Network + blocking | Mininet, Open vSwitch, os-ken OpenFlow controller |
| Live traffic features | TShark + tcpreplay (replays real attack captures) |
| Monitoring | Streamlit live dashboard |
| Quality | 224 automated tests; one config file; repeatable runs |

Ran on: Ubuntu VM (UTM) on a MacBook Pro M5, 4 CPUs, 7 GB RAM, all free and open-source software

# Implementation: Step by Step

**1 Load dataset**
2.2 million labelled packets, 15 classes

→

**2 Fix dataset errors**
label leak (Galal 2026); broken MITM rows

→

**3 Build features**
service port, content, connection behaviour

→

**4 Train classifier**
LightGBM on the server's labelled set

**5 Split into 10 sites**
each site gets very different traffic

→

**6 Federated rounds**
local training + trust check, 20 rounds

→

**7 Evaluate**
accuracy, sabotage tests, unseen-attack tests

→

**8 Live demo**
replay attack → detect → block on switch

# Implementation: How We Raised Accuracy

**The problem:** in web attacks like SQL injection and XSS, only 5–17% of packets carry any content. The rest are identical "acknowledgement" packets, the same for every attack, so the classifier could not tell them apart.

### 1. Which service?

The server port of each connection: web (80), MQTT (1883), etc.

### 2. What was sent?

Numbers describing the content: length, randomness, SQL or script keywords, uploads, logins.

### 3. How did the conversation behave?

Group packets into connections; every packet gets its connection's totals and rates.

**+ rare attack types weighted higher.** We never use IP addresses or timestamps, since those would let the model cheat.

# Results: Dataset and How We Tested

| Edge-IIoTset | Value |
|---|---|
| Packets | 2.2 million |
| After cleaning | 1.99 million |
| Classifier trained on | 1.43 million |
| Tested on (unseen) | 397 thousand |
| Classes | 15 (Normal + 14 attacks) |
| Normal traffic | 71% |
| Rarest attacks | MITM, Fingerprinting |

### A strict test

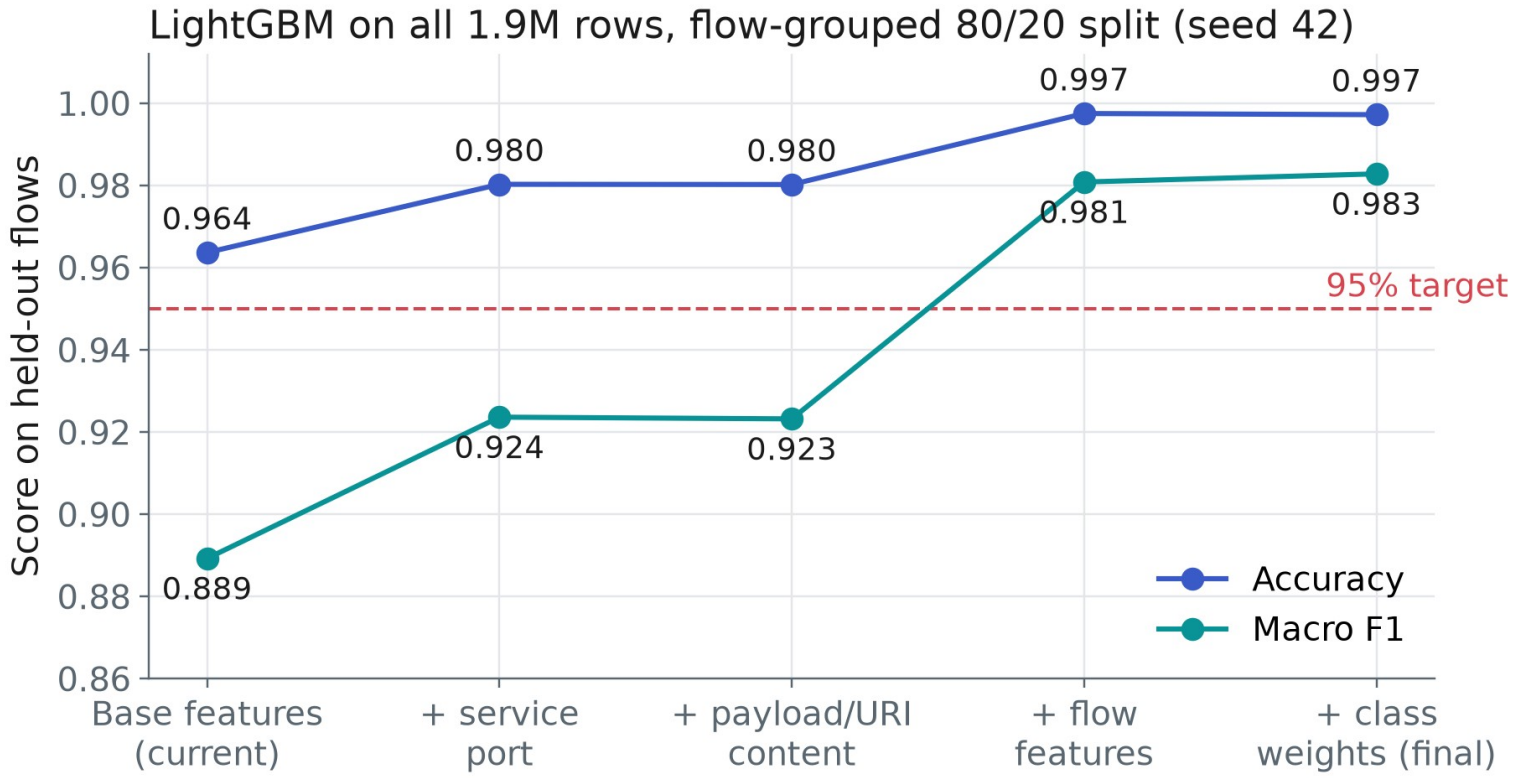
We test on whole connections the model has never seen, so it cannot pass by memorising them.

Repeated with 3 different random seeds.

**Accuracy** = share of correct answers. **Macro F1** = average score over all 15 classes, so rare attacks count as much as common ones.

**Fixed first:** a formatting error in the public dataset gave away the answer (one column alone separated normal from attack perfectly), and the MITM rows were in the wrong columns.

# Results: Accuracy Target Met




**99.7%**

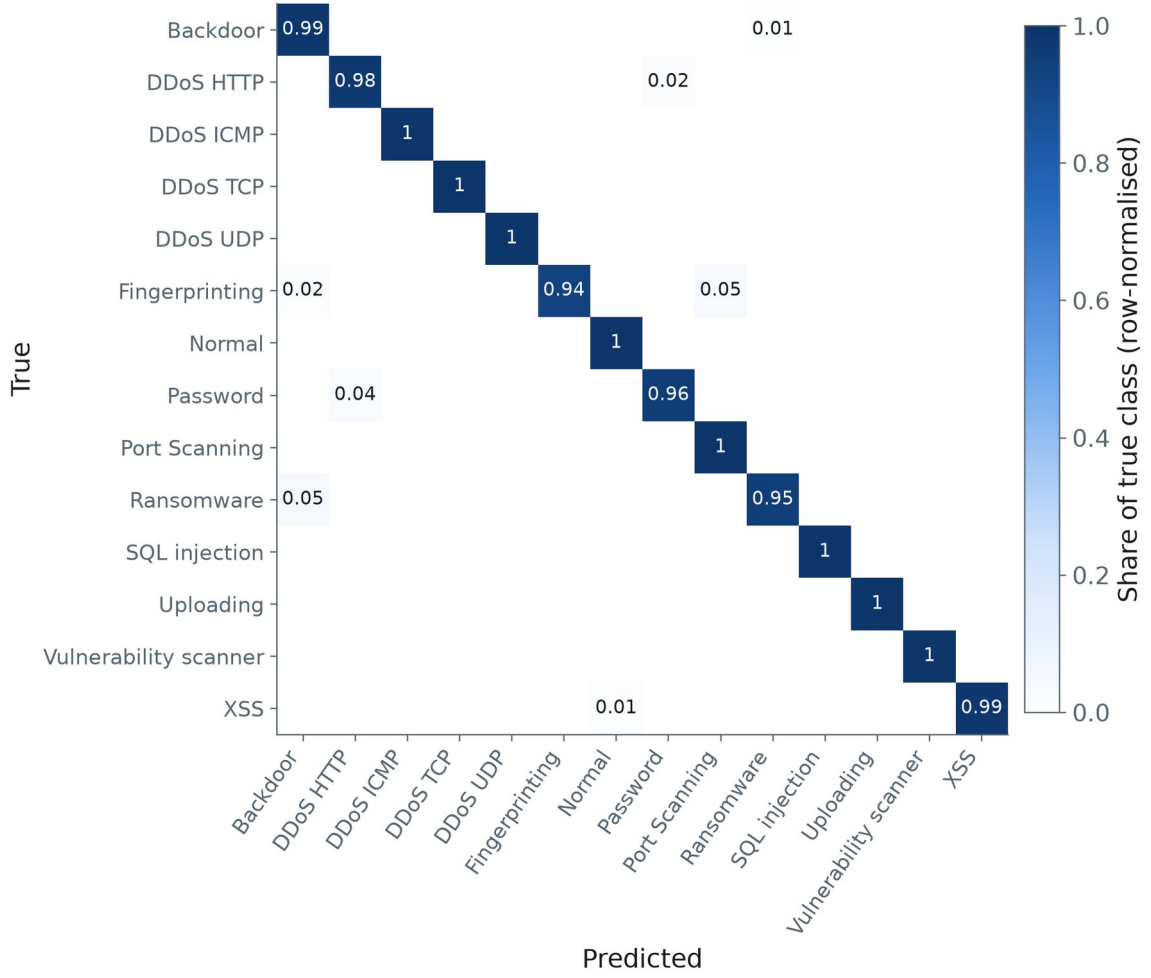
accuracy (was 96.4%)

**0.983**

macro F1 (was 0.889)

Target: ≥ 95% and ≥ 0.95.
Same result with 3 seeds (0.983–0.985).
Still 99.7% / 0.979 when trained on only 5% of the data.

# Results: Confusion Matrix (Final Classifier)




### How to read it

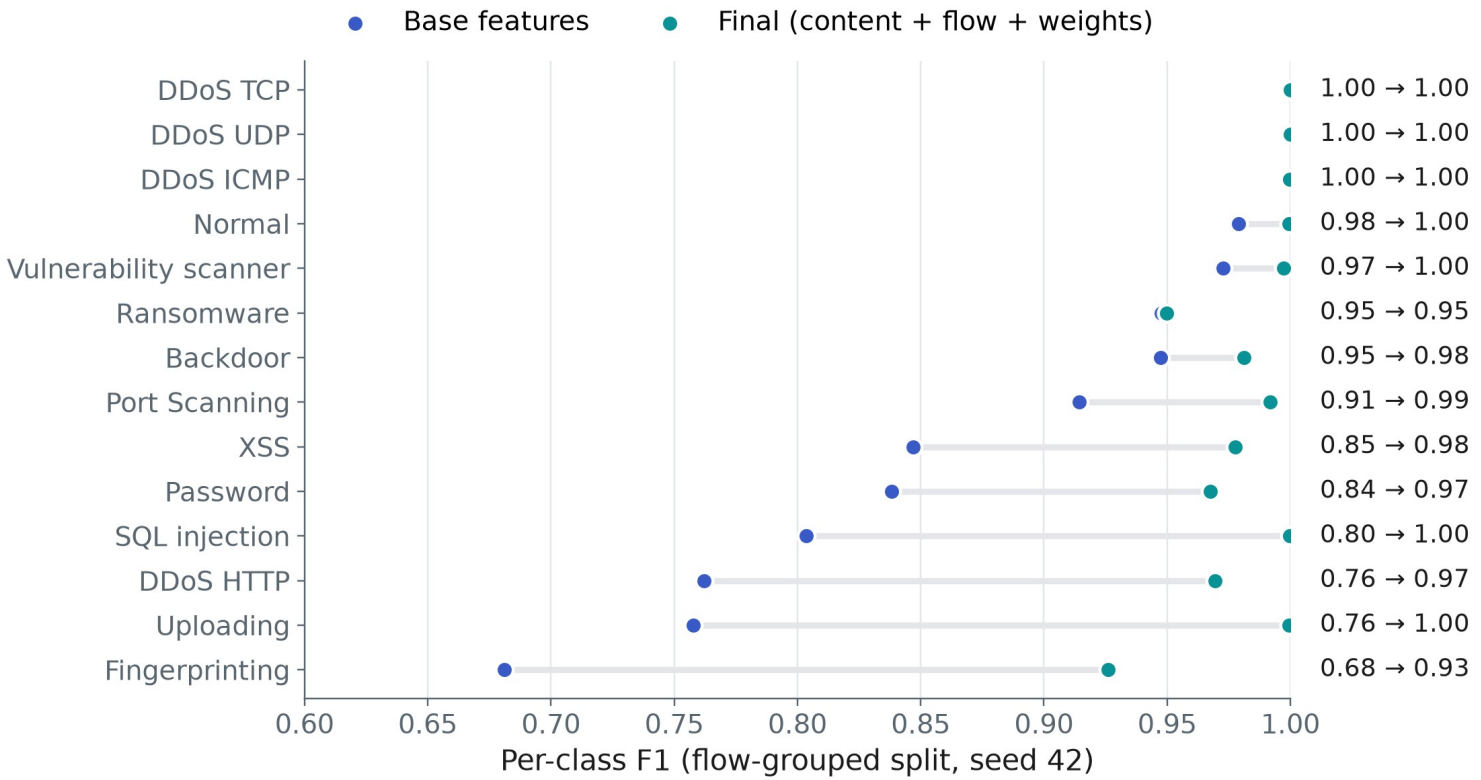
Rows = the real attack, columns = what we predicted. A perfect model is a dark diagonal.

**Almost perfect:** 11 of 14 classes are 0.98–1.00.

**Small mix-ups:** Password ↔ DDoS HTTP (both hit the same web server), Ransomware → Backdoor (same port).

MITM is a single connection, so it can't appear in this strict test (random split: F1 0.99).

# Results: Every Attack Type Improved

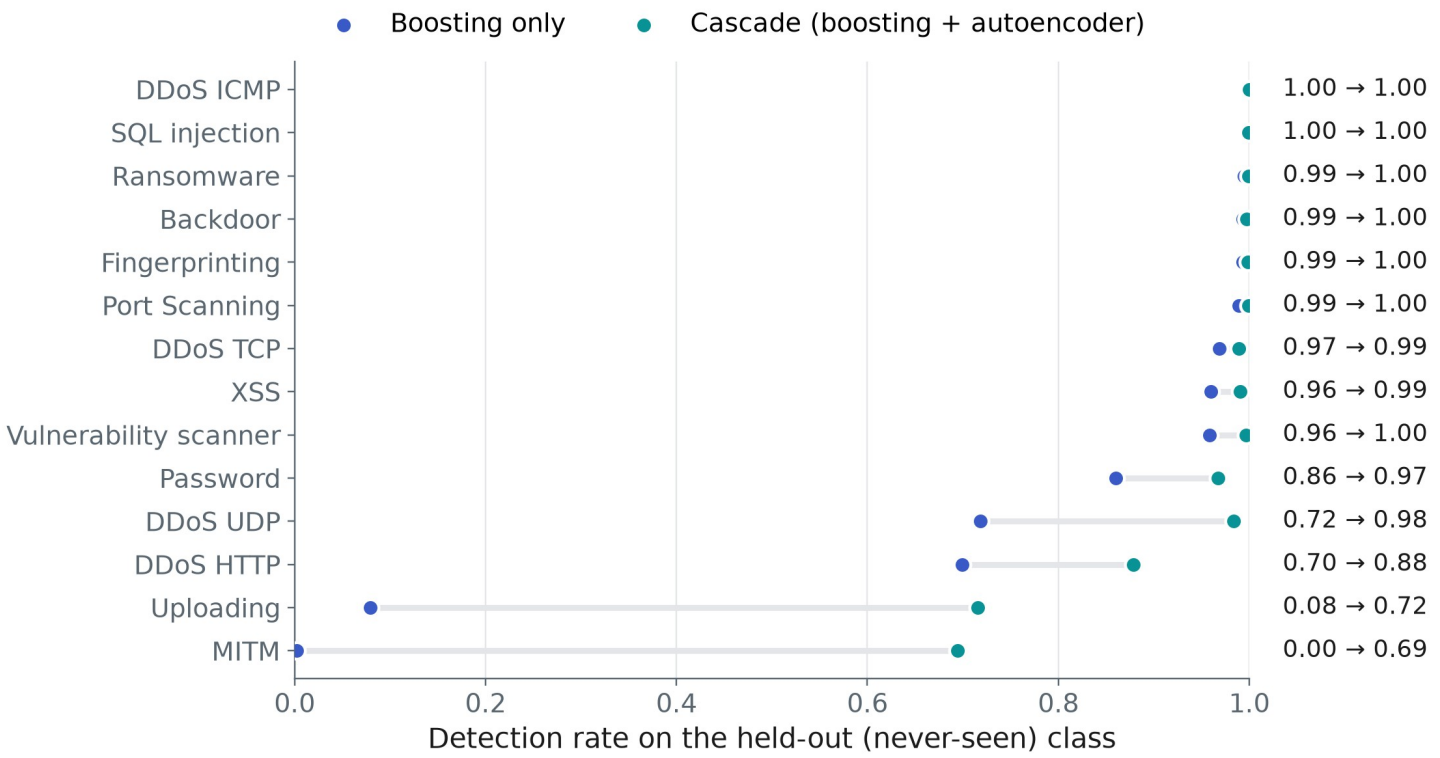



**Biggest gains**

Uploading 0.76 → 1.00
DDoS HTTP 0.76 → 0.97
SQL injection 0.80 → 1.00
Password 0.84 → 0.97
XSS 0.85 → 0.98
Fingerprinting 0.68 → 0.93

Exactly the web and rare attacks that were weak before.

# Results: Catching Attacks Never Seen Before




**0.94**

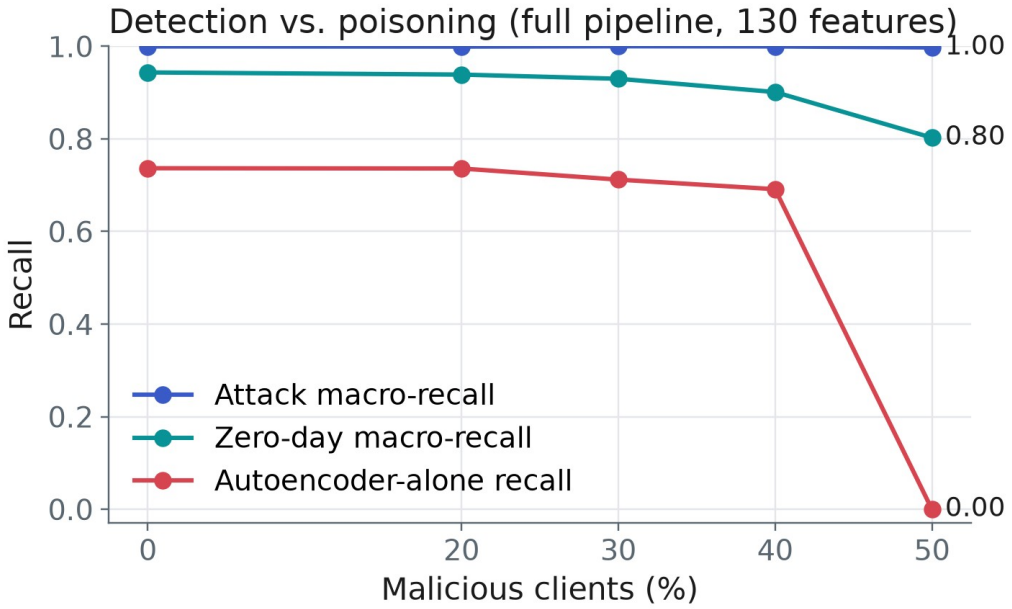
detection with both checks (was 0.74)

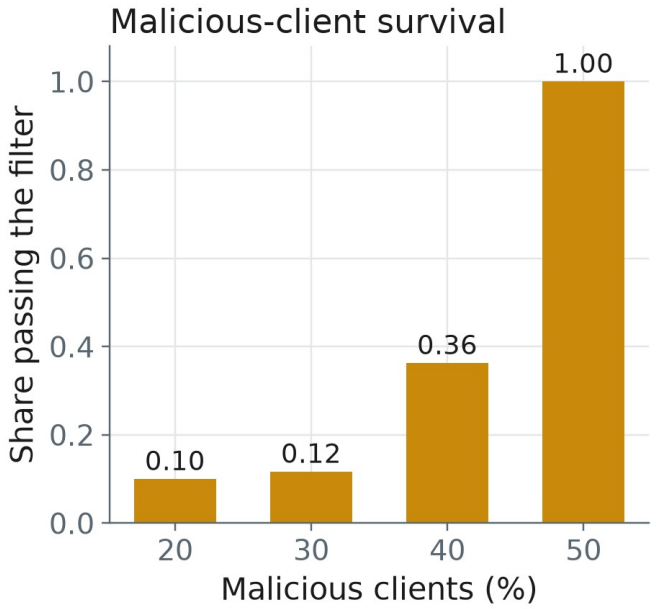
**0.80**

classifier alone (was 0.53)

Each attack type was hidden from training, then tested.
Password 0.44 → 0.97, XSS 0.52 → 0.99.
Worse: MITM 0.89 → 0.69.

# Results: Resisting Sabotage

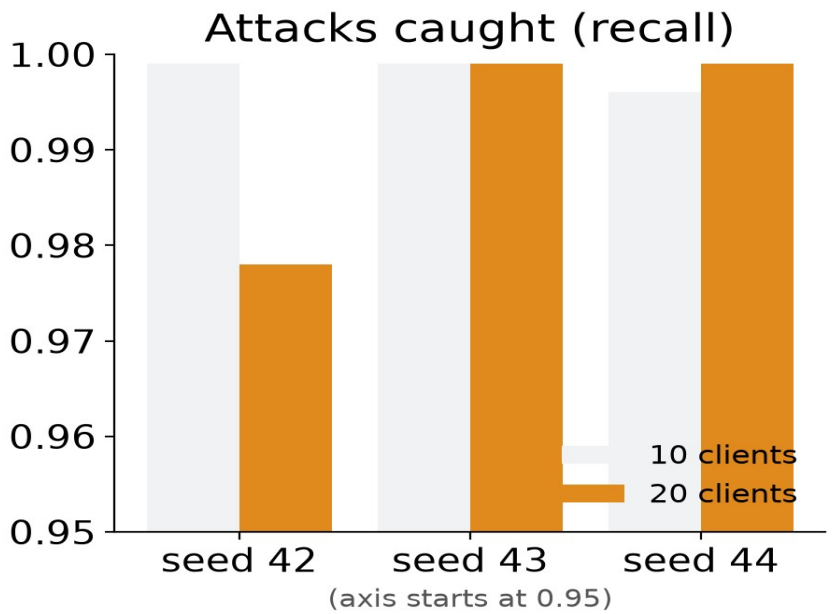


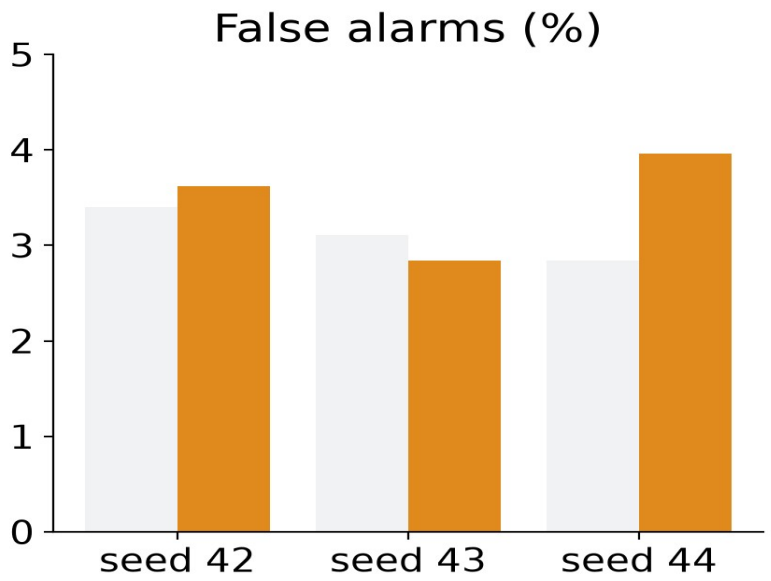




**What it shows**

Detection stays at 99.7–100% with up to 50% malicious sites, and new-attack detection at 90–94% up to 40%. Without the trust check (plain averaging) the anomaly check is ruined at 30%. At 50% attackers become the majority and the check fails, a known limit of this kind of defence.

# Results: Scaling Up and Repeating




**What it shows**

Real runs, not a simulation: 10 and 20 separate client processes, three seeds each. Attacks caught on average: 0.998 with 10 sites and 0.992 with 20; false alarms stay at 2.8–4.0%. One 20-site run (seed 42) dips to 0.978 and is not yet explained.

# Results: Live Demo (Detect and Block)

**1 Replay**
real Backdoor attack capture, 24,914 packets, into the emulated network

→

**2 Detect**
classified as Backdoor with 99.7% confidence

→

**3 Decide**
confidence ≥ 85% → block

→

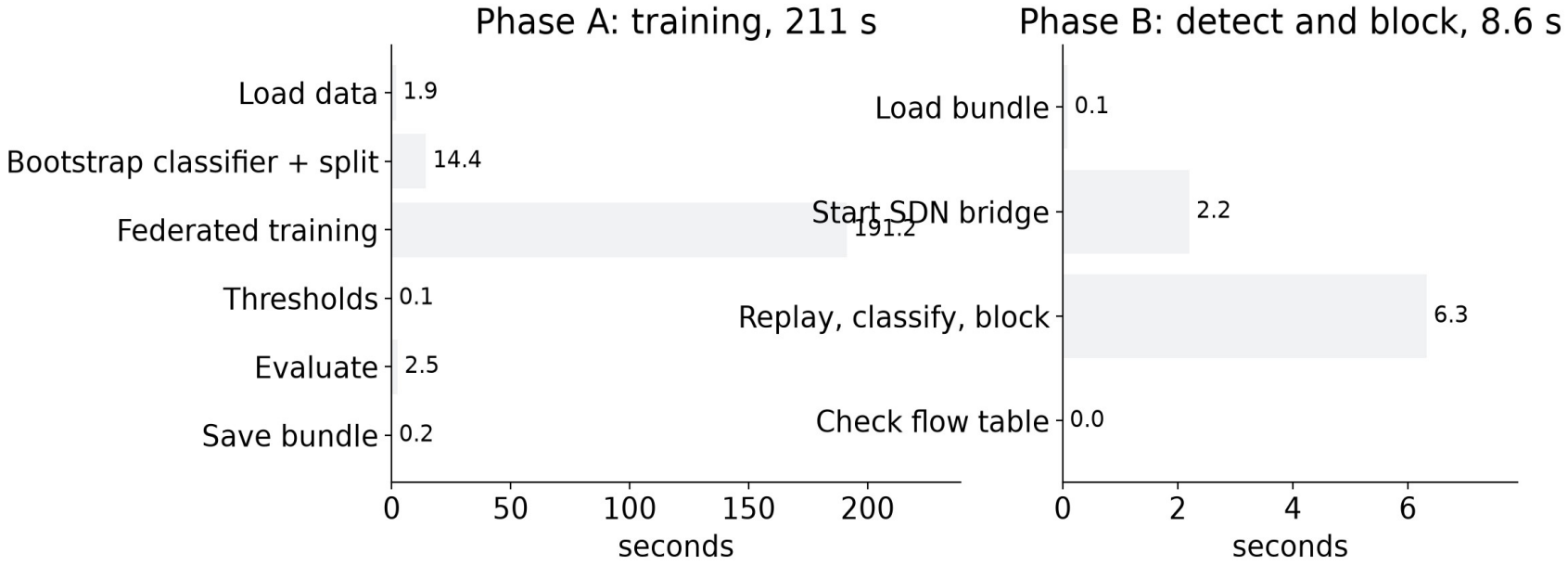
**4 Block**
drop rule installed on the switch and read back to confirm

**Rule on the switch**

priority=100, ip, nw_src=10.0.0.1  actions=drop

Real 10-site federated run: 0.999 attack detection (was 0.889), 3.5% false alarms. 24,669 of 24,914 packets flagged.  Limitation: the decision is made after the capture is analysed, not packet by packet.

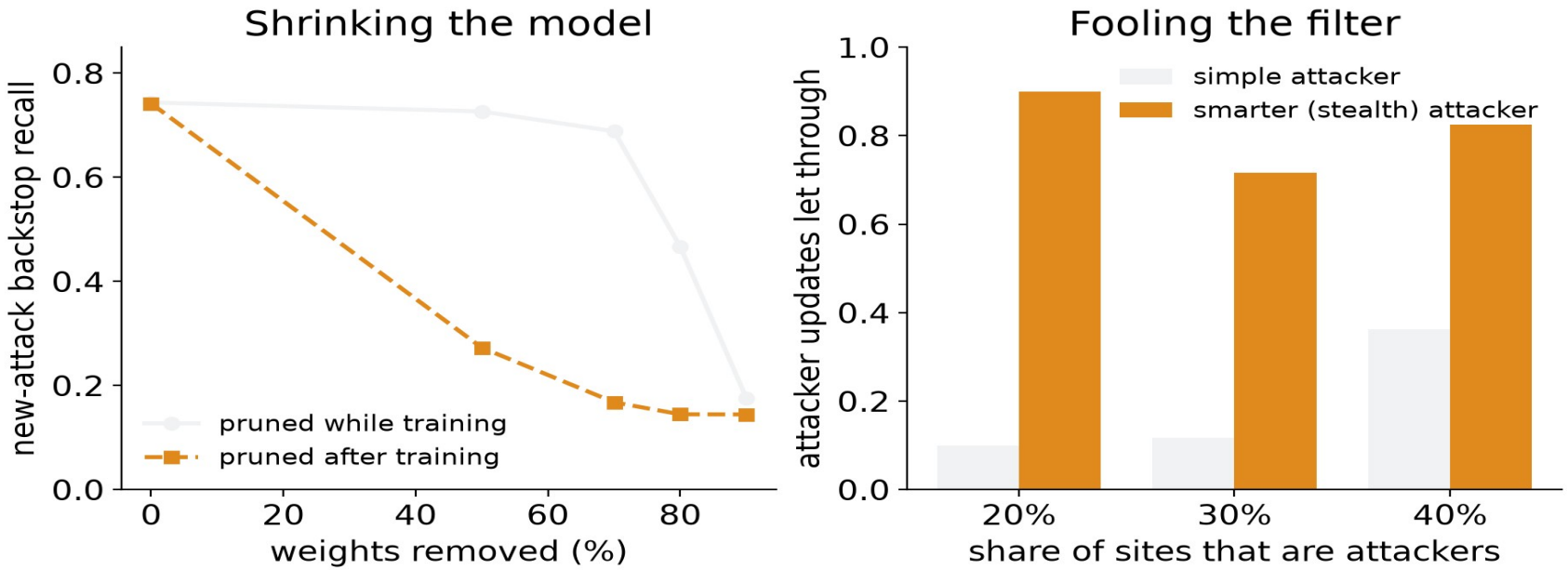
# Results: One Whole Run, Timed




**What it shows**

A complete run, from raw data to a block rule on the switch, took about 219 seconds. Training the shared model is almost all of it; once a model exists, spotting and blocking an attack takes seconds.

# Results: Pruning and Smarter Attackers




**What it shows**

Removing 70% of the detector's weights costs little if done while training (backstop recall 0.74 to 0.69), but ruins it if done afterwards. A smarter attacker slips past the filter most of the time, though known attacks are still caught. Across 3 seeds, new-attack detection stays 0.94–0.95.

# Conclusion

- **Accuracy target met:** 99.7% accuracy and 0.983 macro F1 on 397k unseen packets (was 96.4% / 0.889)
- **Whole system improved:** real 10-site run 0.999 detection (was 0.889)
- **Catches new attacks:** 0.94 vs 0.80 detection of never-seen attack types (was 0.74 / 0.53)
- **Resists sabotage:** steady with up to 40% malicious sites
- **Acts automatically:** detected attacker blocked on the switch, verified
- **Honest data:** found and repaired broken MITM rows; fixed a label leak first reported by Galal (2026)
- **What's novel:** we found no published system combining all five parts, and the MITM repair
- **Project complete:** every planned phase done and verified end to end; paper being written

# Future Enhancements

- Send sparse updates and test pruned models on new attacks (pruning itself is done; communication is not yet reduced)
- Block attacks while they are still happening (decide after the first few packets)
- Test smarter attackers, and more than 50% bad sites
- Repeat every federated experiment with several seeds
- Test on a second dataset that includes normal web traffic
- Deploy on real edge devices and a hardware switch

# Papers Published

- **Status:** paper in preparation (draft in progress)
- **Working title:** FL-IDS: Federated Learning-Based Intrusion Detection System for IoT/IIoT
- **Venue:** not yet submitted

# References


- M. A. Ferrag et al., “Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning,” IEEE Access, vol. 10, pp. 40281–40306, 2022.
- V. Mothukuri et al., “Federated-learning-based anomaly detection for IoT security attacks,” IEEE IoT Journal, vol. 9, no. 4, 2022.
- B. Li et al., “DeepFed: Federated deep learning for intrusion detection in industrial cyber–physical systems,” IEEE TII, vol. 17, no. 8, 2021.
- X. Cao, M. Fang, J. Liu, N. Z. Gong, “FLTrust: Byzantine-robust federated learning via trust bootstrapping,” NDSS, 2021.
- J. Xu et al., “Byzantine-robust federated learning through collaborative malicious gradient filtering (SignGuard),” IEEE ICDCS, 2022.
- H. Zhang et al., “A two-stage intrusion detection method based on LightGBM and autoencoder,” Math. Biosci. Eng., vol. 20, no. 4, 2023.
- Z. Abou El Houda, A. S. Hafid, L. Khoukhi, “MiTFed: … federated learning using SDN and blockchain,” IEEE TNSE, vol. 10, no. 4, 2023.
- J. L. Hernandez-Ramos et al., “Intrusion detection based on federated learning: a systematic review,” ACM Computing Surveys, vol. 57, no. 12, 2025.
- I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” ICISSP, 2018.




# Thank You